

DarkSage

Security Architecture

Version 0.2.2 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: [LightSwornPVP/DarkSage](#)

Source baseline commit: [34a65818d48f596122281075e18c6e5f36ec93b5](#)

Generation date: 2026-07-25T13:31:10-04:00

Document ID: DS-008

Document Control

Document ID	DS-008
Title	Security Architecture
Version	0.2.2
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-24
Last Updated	2026-07-24

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Revision History

Version	Date	Author	Summary
0.1.0	2026-07-24	TheSinnerMan	First controlled draft, authored as part of the Batch 1 grouped pass (DS-007/DS-008/DS-009). Translates SECURITY_RULES.md and DS-002's DS-SEC family, together with DS-004's security-relevant architecture (DS-ARC-015/019/021/022/024) and DS-006's session/authentication contracts (DS-API-COR-004/010), into a consolidated security architecture under the DS-SCA prefix.
0.2.0	2026-07-24	TheSinnerMan	Targeted repair for three independent-audit High findings, added as new §13a (appended after existing content to preserve requirement-ID document order, mirroring the DS-004 §16a repair precedent): H1 added DS-SCA-023 (Protected Assets, Threat Actors, and Trust Boundaries) and DS-SCA-024 (Material Threat Scenarios and Controls, a 10-row scenario/control/fail-safe/residual-risk table) making the threat model actionable, with a forward pointer added to DS-SCA-001. H2 added DS-SCA-025 (Audit-Log Integrity and Protection Architecture) — append-only/tamper-evident writes, authorized writer/reader boundaries, deterministic ordering, fail-closed-on-unavailable-audit behavior — without mandating a specific storage vendor. H3 added DS-SCA-026 (Security Incident Response Lifecycle) — a 10-step detection-through-post-incident-review lifecycle explicitly covering compromised sessions/credentials, suspected unauthorized trading, compromised update/dependency paths, audit-log integrity failures, malicious external data/integrations, and local-device compromise, preserving owner/user authority and requiring stronger validation before live-trading capability is restored. §14 Non-Goals updated to exclude operational-staffing/legal-response policy.

Version	Date	Author	Summary
0.2.1	2026-07-24	TheSinnerMan	Narrow repair for independent-audit finding DS-008-H2 (audit-log integrity verification): added DS-SCA-027 (Audit-Log Integrity Verification Contract, Committed/MVP) defining the deterministic detection behavior DS-SCA-025's append-only architecture required but did not itself specify — protected record set, integrity evidence for modification/reordering/truncation/selective-deletion/unauthorized-insertion, verification triggers (startup, pre-review, scheduled, post-fault), verification result states, fail-closed behavior, escalation into DS-SCA-026's incident lifecycle, evidence-preservation/repair boundaries, and an explicit prohibition on silently rebuilding audit history to hide a verification failure. Implementation-neutral — no vendor or cryptographic product mandated. DS-SCA-025 updated with a one-sentence pointer to DS-SCA-027; DS-SCA-026's audit-log-integrity-failure cross-reference updated to cite DS-SCA-027 as the detection/escalation mechanism.
0.2.2	2026-07-24	TheSinnerMan	Consolidated cleanup pass: added a Traceability note to DS-SCA-019 explicitly cross-referencing it to §13a's threat-scenario table (DS-SCA-024 row 8), protected-assets list, and trust-boundary list (DS-SCA-023), and noting that the update/distribution-mechanism grounding gap is recorded once (Appendix A #2) rather than restated inconsistently. Also recorded, rather than silently closed, that DS-009 currently has no corresponding tamper/signature-verification test for this requirement. No security architecture, fail-closed behavior, or classification was changed.

Table of Contents

Document Control	2
Revision History	3
Table of Contents	5
1. Purpose	7
2. Scope	7
3. Audience	7
4. Definitions	7
5. Threat Model	8
DS-SCA-001 — Threat Model Scope and Fail-Closed Default	8
6. Credential and Secrets Architecture	9
DS-SCA-002 — Credential and Secrets Storage Architecture	9
DS-SCA-003 — AI Provider Credential Handling	9
7. Authentication and Authorization Architecture	9
DS-SCA-004 — Session Lifecycle Security Architecture	9
DS-SCA-005 — Authentication and Authorization Enforcement	10
DS-SCA-006 — Backend-Enforced Security Decisions	10
DS-SCA-007 — Strong Authentication for High-Risk Actions	10
8. AI Output and Input Validation Architecture	11
DS-SCA-008 — AI Output Validation Security Boundary	11
DS-SCA-009 — Input Validation and Injection Defense	11
9. Data Protection and Communication Security	12
DS-SCA-010 — Local Data and Database Security	12
DS-SCA-011 — Secure Communication and Network Exposure Defaults	12
10. Trading Safety Architecture	12
DS-SCA-012 — Order Authorization Trail and Broker Safety	13
DS-SCA-013 — Broker Credential Least Privilege	13
DS-SCA-014 — Broker Reconciliation Security	13
DS-SCA-015 — Fail-Closed Trading Safety Behavior	13
DS-SCA-016 — Emergency Controls Security	14
11. Client and Mobile Security	14
DS-SCA-017 — Desktop and Mobile Client Secret Storage	14
12. Supply Chain, Dependencies, and Update Integrity	15

DS-SCA-018 — Dependency and Supply-Chain Controls	15
DS-SCA-019 — Application Update Integrity	15
13. Audit and Process Controls	16
DS-SCA-020 — Security Event Audit Logging	16
DS-SCA-021 — Independent Review and Development Agent Security Constraints	16
DS-SCA-022 — Security Review Gate Before Live Trading	16
13a. Missing Security Architecture Contracts (added in the DS-008-H1/H2/H3 repair)	17
DS-SCA-023 — Protected Assets, Threat Actors, and Trust Boundaries	17
DS-SCA-024 — Material Threat Scenarios and Controls	18
DS-SCA-025 — Audit-Log Integrity and Protection Architecture	19
DS-SCA-026 — Security Incident Response Lifecycle	20
DS-SCA-027 — Audit-Log Integrity Verification Contract	21
14. Non-Goals	23
15. Dependencies	23
16. Risks and Constraints	23
17. Verification Approach	23
18. References	23
Appendix A — Open Questions	24

1. Purpose

DS-008 is the authoritative security architecture for DarkSage: the threat model, credential/secrets handling, authentication and authorization architecture, broker/trading safety controls, data protection, audit logging, supply-chain and update-integrity controls, and the process-level review requirements that apply to security-critical code. Where DS-002's DS-SEC family and `SECURITY_RULES.md` state *what* must hold, DS-008 states the *architecture* that discharges those obligations, consistent with DS-004's already-approved boundaries and DS-006's already-approved API contracts — DS-008 does not redesign either.

2. Scope

This document governs:

- the threat model and its boundary (desktop, mobile, backend, market-data/broker integrations, AI systems, databases, deployment, development agents, future live trading);
- credential and secrets storage architecture, including AI provider and (future) broker credentials;
- authentication, session lifecycle, and authorization architecture;
- backend-enforcement and input-validation architecture;
- broker/trading safety controls (order authorization, broker safety verification, reconciliation, fail-closed behavior, emergency controls) at the governance/architecture level;
- local data protection, database security, and secure communication;
- dependency, supply-chain, and application-update-integrity controls;
- audit logging and security-event recording; and
- process-level controls (independent review, development-agent constraints, the pre-live-trading security review gate).

DS-008 does not govern: product-level feature commitments (DS-002), general technical architecture outside its security-relevant surface (DS-004 remains authoritative for that), database schema (DS-005), API request/response contracts (DS-006 remains authoritative for all callable client/backend behavior — DS-008 references DS-006's endpoints rather than redefining them), UI presentation (DS-007), or testing procedure (DS-009, which tests against the controls this document defines).

3. Audience

Backend/security contributors, independent auditors, and future Codex authors implementing or extending DarkSage's security controls.

4. Definitions

See DS-001 §24 and DS-002 §4. Additional terms:

Term	Meaning
Fail closed	On uncertainty or failure of a security-relevant determination, the system defaults to blocking the action rather than allowing it (SECURITY_RULES.md Core Security Rule)
Security-critical code	Code implementing the Risk Engine, Broker Adapter/execution, kill switch (Emergency Stop/Flatten), duplicate-order protection, broker reconciliation, authentication, authorization, or secret handling — the set SECURITY_RULES.md "Independent Review" names
Governance boundary	A Committed/MVP constraint on future implementation (e.g., "no endpoint may bypass the pipeline") that is fixed now even though the implementation it constrains is Planned — the same pattern DS-EXE-001/DS-API-EXE-001 already establish

5. Threat Model

DS-SCA-001 — Threat Model Scope and Fail-Closed Default

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Purpose," "Fail Closed," "Core Security Rule"

Description: DarkSage's threat model covers the desktop client, the (future) mobile client, backend services, market-data integrations, broker integrations, AI systems, databases, deployment infrastructure, development agents, and future live trading. Whenever critical uncertainty exists about system state (authentication unavailable, market data stale, broker mismatch, Risk Engine failure, database inconsistency, Permissions Engine unavailable), the system shall block new trades rather than guess. When convenience conflicts with protection of user money, credentials, trading authority, or account access, protection takes priority.

The concrete protected-asset catalog, threat-actor list, trust-boundary list, and material threat-scenario/control table this threat model covers are detailed in §13a (DS-SCA-023/DS-SCA-024, added in the DS-008-H1 repair) rather than restated here, so this section states the model's scope and default posture once, without duplicating the actionable detail.

Acceptance Criteria:

- Every security-relevant determination in this document states its fail-closed behavior explicitly, not by omission.
- No Committed/MVP or Planned requirement in this Codex authorizes a fail-open default for a listed uncertainty condition.

Testing: Fail-closed adversarial test per listed uncertainty condition, once its underlying system exists.

6. Credential and Secrets Architecture

DS-SCA-002 — Credential and Secrets Storage Architecture

Release Classification: Committed / MVP | **Governing Source:** DS-SEC-001 (Committed); SECURITY_RULES.md "Secrets"

Description: DarkSage shall never commit API keys, broker credentials, access/refresh tokens, passwords, private keys, database passwords, cloud credentials, or signing secrets to source control, and shall never expose them in logs, exports, error messages, README files, issues, test fixtures, screenshots, or client-visible output. Production credentials shall use OS-appropriate secure storage (e.g., Windows Credential Manager, macOS Keychain) or an encrypted secrets vault; development may use .env files excluded by .gitignore.

Acceptance Criteria: Matches DS-SEC-001's acceptance criteria exactly; .gitignore excludes, at minimum, .env, .env.*, secret files, credentials, local databases where appropriate, build artifacts, dependency directories, and private keys (SECURITY_RULES.md "Git Security").

Testing: Log/export/secret-scanning test confirming absence of secret values across Committed/MVP surfaces (docs/secret-scanning.md), shared with DS-SEC-001's own test.

DS-SCA-003 — AI Provider Credential Handling

Release Classification: Committed / MVP | **Governing Source:** DS-ARC-015 (Committed); SECURITY_RULES.md "AI Privacy and Provider Credentials"

Description: This requirement restates DS-ARC-015's architecture as a security-architecture control rather than redefining it: AI provider API keys are never committed, logged, or exposed in frontend/client source; production keys use OS credential storage or an encrypted vault; a key is never sent to a provider other than the one the user configured it for; and a stored key is never redisplayed in full once saved. Local and cloud AI shall not receive credentials or unnecessary sensitive account data.

Acceptance Criteria: Matches DS-ARC-015's acceptance criteria exactly.

Testing: Secret-scanning test; credential-exposure audit across frontend build output (shared with DS-ARC-015's own test).

7. Authentication and Authorization Architecture

DS-SCA-004 — Session Lifecycle Security Architecture

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Session Management"; DS-API-COR-010 (DS-006, Committed)

Description: This requirement is the security-architecture instantiation of DS-API-COR-010, which DS-008 does not redefine: sessions support expiration, logout, revocation (one or all), secure refresh, and device/session visibility, exposed through POST /auth/sessions,

GET/DELETE /auth/sessions/current, POST /auth/sessions/refresh, GET /auth/sessions, DELETE /auth/sessions/{session_id}, and DELETE /auth/sessions (DS-006 §5, DS-API-COR-010). DS-008's architectural obligation is that every issued session credential is backend-enforced, expiration is never client-trusted, and revocation takes effect without requiring a backend restart or cache expiry.

Acceptance Criteria: Matches DS-API-COR-010's acceptance criteria exactly; no architectural component may treat an expired or revoked credential as valid due to caching, regardless of cache layer.

Testing: Session-lifecycle test matrix (shared with DS-API-COR-010's own test, DS-006 §5).

DS-SCA-005 — Authentication and Authorization Enforcement

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Authentication and Authorization"; DS-API-COR-004 (DS-006, Committed)

Description: Every non-public backend endpoint requires authentication, enforced on the backend. Authorization is evaluated separately from authentication against permission groups appropriate to the endpoint's sensitivity: read-only, trade approval, Auto-Trader control, administrative settings, and live-trading management, per SECURITY_RULES.md's permission-group list. This requirement is the security-architecture instantiation of DS-API-COR-004 and does not redefine its contract.

Acceptance Criteria: Matches DS-API-COR-004's acceptance criteria exactly; an authenticated-but-unauthorized request receives 403, distinct from an unauthenticated request's 401.

Testing: Authentication/authorization test matrix per permission group (shared with DS-API-COR-004's own test).

DS-SCA-006 — Backend-Enforced Security Decisions

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Backend Enforcement"; DS-ARC-001 (Committed)

Description: Security decisions (authentication, authorization, risk determinations, permission checks) are enforced on the backend; a UI-only control is never sufficient. This is the security-architecture restatement of DS-ARC-001's client/server boundary: no client, desktop or mobile, is ever the sole enforcement point for a security-relevant decision.

Acceptance Criteria:

- No security-relevant check exists only in client-side code without an equivalent backend-enforced check.
- A client bypassing or disabling its own UI-level check does not gain any capability the backend would otherwise deny.

Testing: Adversarial test: bypass or disable a client-side check directly (e.g., via API call) and confirm the backend independently rejects the action.

DS-SCA-007 — Strong Authentication for High-Risk Actions

Release Classification: Committed / MVP (governance boundary); Planned (implementation, tied to each triggering action's own phase) | **Governing Source:** SECURITY_RULES.md "Strong Authentication"

Description: High-risk actions — enabling live trading, changing live broker credentials, increasing major risk limits, Emergency Flatten, and re-enabling trading after a security event — require strong authentication. This obligation is fixed now as a Committed governance boundary (mirroring DS-EXE-001/DS-API-EXE-001's pattern): no future endpoint or UI surface implementing any of these actions may ship without a strong-authentication check, even though the actions themselves (live trading, Emergency Flatten) remain Planned, Phase 7/13.

Acceptance Criteria:

- Every future implementation of a listed high-risk action is reviewed against this requirement before merge.
- No listed action is reachable through a code path requiring only standard session authentication (DS-SCA-005) once implemented.

Testing: Requirements/design review for each future high-risk-action implementation (mirrors DS-EXE-001's own test pattern).

8. AI Output and Input Validation Architecture

DS-SCA-008 — AI Output Validation Security Boundary

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-011 (Committed); SECURITY_RULES.md "AI Output Is Untrusted Input"

Description: This requirement is the architecture-level instantiation of DS-PRD-011, which DS-008 does not redefine: structured output from any AI provider (local or cloud) is validated before any application use or state transition, and shall never directly execute shell commands, modify security settings, submit broker orders, change live credentials, or override risk controls. Validation fails closed — a validation failure rejects/ignores the output rather than applying it with a warning.

Acceptance Criteria: Matches DS-PRD-011's acceptance criteria exactly.

Testing: Adversarial test crafting AI output attempting a disallowed action, confirming rejection before any sensitive code path (shared with DS-PRD-011's own test).

DS-SCA-009 — Input Validation and Injection Defense

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Input Validation," "Common Application Security"

Description: All external input — user input, market-data APIs, broker APIs, AI output, imported files, strategy definitions, and mobile/desktop client requests — is validated at the trust boundary where it enters the backend. DarkSage uses parameterized database queries and safe frontend rendering, protecting against SQL injection, command injection, cross-site scripting, CSRF (where applicable), unsafe deserialization, and unvalidated external data.

Acceptance Criteria:

- No code path constructs a database query by direct string concatenation of external input.
- Every listed external input source has a documented validation step before use.

Testing: Injection-class security test suite (SQL injection, command injection, XSS, CSRF, deserialization) across every external input source (SECURITY_RULES.md "Security Testing").

9. Data Protection and Communication Security

DS-SCA-010 — Local Data and Database Security

Release Classification: Committed / MVP | **Governing Source:** DS-DAT-001 (Committed); SECURITY_RULES.md "Database and Backups"

Description: Databases are never publicly exposed. Production databases use authentication, restricted network access, least-privilege accounts, and encryption where appropriate. Critical data has secure backups and a recovery procedure. This requirement applies DS-DAT-001's local-first storage obligation with the security controls appropriate to it.

Acceptance Criteria:

- No Committed/MVP deployment configuration binds the database to a publicly reachable interface by default.
- A documented backup/recovery procedure exists for critical local data categories (DS-DAT-001).

Testing: Network-exposure scan confirming the database is not publicly reachable; backup/recovery drill.

DS-SCA-011 — Secure Communication and Network Exposure Defaults

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Secure Communication," "Network Exposure"; DS-ARC-018 (Committed)

Description: Production remote communication uses encrypted transport (HTTPS, TLS, secure WebSockets where used); localhost HTTP is acceptable for local development (DS-ARC-018 Stage 1). Local development services are not publicly exposed by default — no automatic router-port opening, public tunnel creation, firewall disabling, or binding sensitive services to all interfaces.

Acceptance Criteria:

- No Committed/MVP production deployment path serves API traffic over unencrypted HTTP.
- No default configuration exposes a local development service beyond localhost without explicit user action.

Testing: Transport-security scan (production configuration); default-configuration network-exposure audit.

10. Trading Safety Architecture

DS-SCA-012 — Order Authorization Trail and Broker Safety

Release Classification: Committed / MVP (governance boundary); Planned (implementation, Phase 7) | **Governing Source:** SECURITY_RULES.md "Order Authorization," "Broker Safety"; DS-EXE-001, DS-API-EXE-001 (DS-006, both Committed)

Description: This requirement restates, at the security-architecture layer, the boundary DS-EXE-001/DS-API-EXE-001 already fix: every order requires an internal authorization trail (authenticated user/system action, allowed strategy, allowed instrument, Risk Engine approval, Permissions Engine approval, execution approval) with no direct client-to-broker bypass. Before any live order is allowed, the system verifies correct broker environment, correct account, correct execution mode, valid credentials, and expected account identifier, protecting against accidentally connecting live code to the wrong account.

Acceptance Criteria: Matches DS-EXE-001/DS-API-EXE-001's acceptance criteria; the broker-environment/account verification check is fixed now per SECURITY_RULES.md, implementation deferred to Phase 7.

Testing: Requirements review (shared with DS-EXE-001's own test, now); broker-safety verification test once implemented (Phase 7).

DS-SCA-013 — Broker Credential Least Privilege

Release Classification: Planned, Phase 7 | **Governing Source:** SECURITY_RULES.md "Least Privilege," "Environment Separation"

Description: Broker access permits only what is necessary; DarkSage shall never require withdrawal permission, external money transfers, or account-ownership changes. Paper and live credentials remain always separate, with separate environments maintained for development, testing, paper trading, and production/live trading.

Acceptance Criteria:

- No broker integration requests a permission scope beyond trading and read access.
- Paper and live credential storage never share the same stored secret or configuration entry.

Testing: Broker permission-scope audit; paper/live credential-separation test (Phase 7).

DS-SCA-014 — Broker Reconciliation Security

Release Classification: Planned, Phase 7 | **Governing Source:** SECURITY_RULES.md "Broker Reconciliation"; DS-ARC-021 (Planned)

Description: DarkSage compares internal and broker state regularly (cash, buying power, positions, open orders, filled orders, average entry price). If reconciliation fails, the system pauses new trading, alerts the user, and records an audit event.

Acceptance Criteria: Matches SECURITY_RULES.md's reconciliation-failure behavior exactly; reconciliation failure never results in silently continuing to trade.

Testing: Simulated reconciliation-mismatch test confirming pause/alert/audit behavior (Phase 7).

DS-SCA-015 — Fail-Closed Trading Safety Behavior

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Fail Closed"; DS-RSK-001 (Committed)

Description: This requirement is the trading-specific instantiation of DS-SCA-001: when authentication is unavailable, market data is stale, a broker mismatch is detected, the Risk Engine fails, the database is inconsistent, or the Permissions Engine is unavailable, DarkSage blocks new trades rather than guessing. This governs the product now, even before the execution pipeline exists (Phase 7), mirroring DS-RSK-001's own fail-safe edge case.

Acceptance Criteria:

- No listed uncertainty condition, once its underlying system exists, results in a trade being allowed to proceed.
- Fail-safe-on-unavailability is tested for the Risk Engine now (shared with DS-RSK-001's own test) and extended to the remaining conditions as their systems are built.

Testing: Fail-safe-on-unavailability test (shared with DS-RSK-001's own test); adversarial uncertainty-injection test per condition once its system exists.

DS-SCA-016 — Emergency Controls Security

Release Classification: Planned, Phase 7 | **Governing Source:** SECURITY_RULES.md "Emergency Controls"; DS-ARC-022 (Planned)

Description: Emergency Stop is easy for authorized users to trigger; Emergency Flatten is more dangerous and requires stronger authentication (DS-SCA-007) and explicit confirmation in live mode. Both remain reachable from any authorized client independent of the normal order-submission code path, per DS-ARC-022, so a partial system failure that would justify triggering them does not also disable them.

Acceptance Criteria: Matches DS-ARC-022's acceptance criteria exactly.

Testing: Independent-reachability test; Emergency Flatten strong-authentication test (shared with DS-ARC-022's own test, Phase 7).

11. Client and Mobile Security

DS-SCA-017 — Desktop and Mobile Client Secret Storage

Release Classification: Committed / MVP for desktop; Planned, Phase 9 for mobile-specific controls | **Governing Source:** SECURITY_RULES.md "Desktop and Mobile Security"; DS-ARC-024 (Planned); DS-SCA-002

Description: Desktop and mobile clients avoid storing broker secrets directly, using secure platform storage where sensitive local data is required — the desktop obligation is already covered by DS-SCA-002 (Committed). This requirement adds the mobile-specific architecture DS-ARC-024 establishes: iOS Keychain for secure storage, and strong authentication (DS-SCA-007) for high-risk mobile actions (pause Auto-Trade, emergency stop, trade approvals), once the mobile client is built.

Acceptance Criteria: Matches DS-ARC-024's acceptance criteria exactly for the mobile-specific portion; the desktop portion is already satisfied by DS-SCA-002.

Testing: Deferred to Phase 9 for the mobile-specific portion (shared with DS-ARC-024's own test).

12. Supply Chain, Dependencies, and Update Integrity

DS-SCA-018 — Dependency and Supply-Chain Controls

Release Classification: Committed / MVP (process-level) | **Governing Source:** SECURITY_RULES.md "Dependency Security"

Description: Dependencies are necessary, maintained where possible, version-controlled, and reviewed before adoption. DarkSage avoids unnecessary dependencies and uncontrolled automatic upgrades in critical areas (security-critical code, per DS-SCA §4 definition).

Acceptance Criteria:

- A new dependency addition to a security-critical area is reviewed before merge, not auto-adopted.
- Dependency-scanning tooling runs as part of the development/test pipeline.

Testing: Dependency-scanning test (SECURITY_RULES.md "Security Testing").

DS-SCA-019 — Application Update Integrity

Release Classification: Planned | **Governing Source:** SECURITY_RULES.md "Core Security Rule," "Dependency Security" (general principle only — no DS-002/DS-004 requirement currently commits a specific update/distribution mechanism; see Appendix A)

Description: Once DarkSage has a distribution/update mechanism (e.g., an Electron auto-updater or installer-based distribution), updates shall be delivered over secure transport (DS-SCA-011) and shall be integrity/signature-verified before install, consistent with the fail-closed principle (DS-SCA-001): an update that fails integrity verification is rejected, not installed with a warning.

Traceability: This requirement is the governing reference for §13a's threat-scenario table (DS-SCA-024, row 8 — "Application integrity / Update-distribution channel"), which already cites DS-SCA-019 as its Required Control basis; §13a's protected-assets list (DS-SCA-023) and trust-boundary list both name "update mechanism ↔ installed application (DS-SCA-019)" as the asset/boundary this requirement protects. The grounding gap this requirement carries — no DS-002/DS-004 requirement yet commits a specific update/distribution mechanism — is recorded once, in Appendix A Open Question #2, rather than restated inconsistently elsewhere; every other cross-reference to DS-SCA-019 in this document points here for that context instead of re-describing the gap.

Acceptance Criteria: Deferred pending selection of a distribution/update mechanism; the secure-transport and signature-verification obligations are fixed now.

Testing: Not yet applicable — deferred to the update mechanism's own implementation. Once a mechanism is selected, DS-009 (Testing and QA) is expected to add a corresponding tamper/signature-verification test alongside DS-SCA-024 row 8's other adversarial coverage; no such test currently exists in DS-009, a gap this repair records rather than silently closes.

13. Audit and Process Controls

DS-SCA-020 — Security Event Audit Logging

Release Classification: Committed / MVP | **Governing Source:** DS-OPS-001, DS-OPS-002 (both Committed); SECURITY_RULES.md "Logging"

Description: This requirement is the security-specific instantiation of DS-OPS-001/002, which DS-008 does not redefine: security-sensitive actions are audited, including login, failed login, credential change, live-trading enable, Auto-Trader enable/disable, risk-limit changes, Emergency Stop, Emergency Flatten, trade override, and permission changes. No log entry ever contains a full secret; sensitive values are redacted (DS-SCA-002).

Acceptance Criteria: Matches DS-OPS-001/002's acceptance criteria; every listed security-sensitive action produces a correlated audit entry once its underlying feature exists.

Testing: Audit-content audit per listed security-sensitive action category, including a secret-redaction check (shared with DS-OPS-001/002's own test).

DS-SCA-021 — Independent Review and Development Agent Security Constraints

Release Classification: Committed / MVP (process-level) | **Governing Source:** SECURITY_RULES.md "Independent Review," "Automated Tooling Security"

Description: No single agent — human or AI — both implements and solely approves security-critical code (DS-008 §4 definition: Risk Engine, broker execution, kill switch, duplicate-order protection, broker reconciliation, authentication, authorization, secret handling). Development agents must follow the repository's governing documents (PROJECT_SPEC.md, ARCHITECTURE.md, ROADMAP.md, AGENTS.md, SECURITY_RULES.md, TRADING_RULES.md) and must not commit secrets, disable security checks, enable live trading without explicit approval, expose services publicly by default, install unnecessary dependencies, or make destructive system changes without approval.

Acceptance Criteria:

- No merge touching security-critical code is approved solely by the same agent/author that authored it.
- No development-agent action in this repository's own history violates a listed constraint (self-referential: this Codex's own separation between Author/Repair Agent and independent auditor, per .ai-workflow/AGENT_PROTOCOL.md, is the process instantiation of this exact requirement for documentation changes, and the same principle extends to code).

Testing: Code-review process audit confirming independent-approval evidence for security-critical merges.

DS-SCA-022 — Security Review Gate Before Live Trading

Release Classification: Committed / MVP (governance boundary); Planned (implementation, Phase 13) | **Governing Source:** DS-EXE-007 (Committed); SECURITY_RULES.md "Security Review Before Live Trading"

Description: This requirement states the security-specific content of the independent security review DS-EXE-007's live-trading gate already requires, without redefining that gate: authentication, authorization, secrets, broker credentials, broker permissions, Risk Engine, execution engine, emergency controls, reconciliation, logging, deployment, mobile controls, and network exposure are each reviewed before live trading is enabled.

Acceptance Criteria: Matches DS-EXE-007's acceptance criteria; the review checklist above is the security-specific detail behind DS-EXE-007's "independent security review has passed" prerequisite.

Testing: Requirements/governance review now (shared with DS-EXE-007's own test); full security review execution deferred to Phase 13.

13a. Missing Security Architecture Contracts (added in the DS-008-H1/H2/H3 repair)

The following close completeness gaps identified by independent audit: DS-SCA-001's threat model referenced a scope without an actionable asset/actor/scenario catalog (H1); DS-SCA-020 defined redaction but not audit-log trustworthiness/tamper-resistance (H2); and no requirement defined the security-incident lifecycle (H3).

DS-SCA-023 — Protected Assets, Threat Actors, and Trust Boundaries

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Purpose"; DS-SCA-001

Description: This requirement enumerates, as a fixed architectural catalog, the assets, actors, and trust boundaries DS-SCA-001's threat model covers, making it actionable rather than a scope statement alone.

Protected assets: user identity/session data (DS-SCA-004/005); broker credentials and integration secrets (DS-SCA-002/003/013); financial/portfolio data (DS-DAT-001, DS-SCA-010); trading permissions and execution authority (DS-SCA-005/012); Risk Engine and validation authority (DS-RSK-001, DS-SCA-012); audit evidence (DS-SCA-020, DS-SCA-025); model inputs/outputs and retrieved evidence (DS-PRD-002, DS-SCA-008); local configuration and application integrity (DS-SCA-018/019).

Threat actors: unauthenticated remote attacker; malicious or compromised authenticated user/session; malware or compromised local process; compromised dependency/update channel; compromised external provider/broker integration; insider or over-privileged process; prompt/data-injection source (market data, AI output, imported files — where applicable).

Trust boundaries: desktop client ↔ backend/service layer (DS-ARC-001, DS-SCA-006); backend ↔ database (DS-SCA-010); backend ↔ broker/provider adapters (DS-ARC-006, DS-EXE-006); Sage/model runtime ↔ deterministic services (DS-PRD-004, DS-SCA-008); local storage ↔ application process (DS-SCA-002); update mechanism ↔ installed application (DS-SCA-019); paper-trading ↔ live-trading boundary (DS-EXE-007, DS-SCA-022).

Acceptance Criteria:

- Every listed asset has at least one governing DS-SCA/DS-002 requirement cited.
- Every trust boundary is enforced by a backend-side check (DS-SCA-006), never a client-side-only control.
- This catalog is reviewed and updated whenever a new asset, actor class, or trust boundary is introduced elsewhere in the Codex.

Testing: Design/architecture-review checklist confirming every new feature introducing an asset, actor, or trust boundary is added to this catalog before merge.

DS-SCA-024 — Material Threat Scenarios and Controls

Release Classification: Committed / MVP | **Governing Source:** DS-SCA-023; SECURITY_RULES.md (cited per scenario)

Description: The following material threat scenarios instantiate DS-SCA-023's assets/actors/boundaries into concrete, testable architectural requirements. Each scenario states the asset at risk, entry point, threat/action, required control, expected fail-safe/fail-closed behavior, residual risk/disposition, and governing reference. This table is architectural — it does not prescribe implementation-specific tooling.

#	Asset at Risk	Entry Point	Threat / Action	Required Control	Fail-Safe / Fail-Closed Behavior	Residual Risk / Disposition	Governing Reference
1	Session/identity data	Public API surface	Unauthenticated remote attacker attempts to reach a non-public endpoint	Authentication required on every non-public endpoint	Reject with 401; no partial access	Low — accepted, standard auth boundary	DS-SCA-005
2	Session/identity data	Stolen/leaked session credential	Compromised authenticated session reused by an attacker	Session lifecycle (expiration, revocation, device visibility)	Revoked/expired session rejected regardless of client-side state	Low — mitigated by DS-SCA-004; residual window bounded by expiration	DS-SCA-004
3	Broker credentials	Credential storage or transit	Malware/compromised local process attempts to read stored broker secrets	OS-appropriate secure storage; never plaintext; never logged	Read attempt outside the secure-storage API fails; no plaintext fallback exists	Low — bounded by OS credential-store guarantees	DS-SCA-002/003/013
4	Trading/execution authority	API request to an execution-adjacent endpoint	Compromised session or malicious client attempts to bypass the TradeValidationPipeline	Backend-enforced, unconditional pipeline boundary (no endpoint may skip a stage)	Request rejected before reaching the Broker Adapter; attempt logged	Low — governance boundary is Committed and reviewed on every new endpoint	DS-SCA-012, DS-EXE-001, DS-API-EXE-001
5	Risk Engine authority	Sage-issued instruction or narrated workaround	Attempt to have Sage or any caller override a Risk Engine determination	Risk Engine authority independent of all callers; fail-safe on unavailability	Determination is not overridden; unavailability blocks rather than allows	Low — adversarial-tested per DS-QA-006	DS-RSK-001, DS-SCA-015
6	Audit evidence	Direct database/file access	Insider or compromised process attempts to truncate, rewrite, or selectively delete audit records	Append-only/immutable-evident audit architecture	Write path rejects modification of existing records; deletion outside retention policy is blocked	Low-Medium — depends on underlying storage guarantees at implementation time	DS-SCA-025

#	Asset at Risk	Entry Point	Threat / Action	Required Control	Fail-Safe / Fail-Closed Behavior	Residual Risk / Disposition	Governing Reference
7	Model inputs/outputs	AI provider response	AI output attempts to trigger a disallowed action (shell command, risk override, credential change)	AI output validation, fail-closed	Output rejected before reaching any sensitive code path	Low — adversarial-tested per DS-QA-008	DS-SCA-008
8	Application integrity	Update/distribution channel	Compromised dependency or update channel delivers a tampered build	Signed/integrity-verified updates over secure transport	Update failing verification is rejected, not installed with a warning	Medium — residual risk until an update mechanism is selected (DS-SCA-019, Appendix A)	DS-SCA-019
9	Broker/portfolio data	Broker adapter response	Compromised or malfunctioning external broker/provider integration returns inconsistent state	Broker reconciliation; provider abstraction isolates a single provider's failure	Reconciliation failure pauses new trading, alerts, and logs; a failing provider does not corrupt internal state	Medium — depends on provider reliability; mitigated, not eliminated	DS-SCA-014, DS-ARC-006
10	Live-trading authority	Strong-auth-gated action	Attempt to enable live trading or perform a high-risk action without strong authentication	Strong authentication required for every listed high-risk action	Action blocked without the strong-auth tier, regardless of session validity	Low — governance boundary fixed now	DS-SCA-007, DS-SCA-022

Acceptance Criteria:

- Every scenario's Required Control has a corresponding DS-SCA/DS-QA testable requirement.
- A new material threat scenario discovered during implementation is added to this table, not handled as an undocumented ad hoc fix.
- No scenario's Residual Risk/Disposition is left blank — an unmitigated risk is explicitly acknowledged, not silently omitted.

Testing: Scenario-to-test traceability audit: each row's Required Control maps to at least one test in docs/traceability/TRACEABILITY_MATRIX.csv or this Codex's DS-QA test set (DS-QA-013).

DS-SCA-025 — Audit-Log Integrity and Protection Architecture

Release Classification: Committed / MVP | **Governing Source:** DS-OPS-001, DS-OPS-002 (both Committed); DS-SCA-020; DS-API-OPS-001 (DS-006, Committed)

Description: This requirement defines the trustworthiness architecture for audit records DS-SCA-020/DS-OPS-002 require, beyond redaction alone. Audit records are append-only (or an equivalent tamper-evident mechanism): once written, an existing record is never modified or deleted by normal application code paths; a correction is recorded as a new, linked entry, never an edit to the original. Authorized writers are limited to the backend components that generate the audited event (never a client, never a direct database write from outside the audit-write path).

Authorized readers are limited to the permission groups DS-SCA-005 defines (at minimum, administrative settings); reads of the audit log are themselves logged where the audit surface is security-sensitive (GET /audit-log, DS-API-OPS-001). Every record carries a timestamp and a deterministic ordering (e.g., a monotonically increasing sequence or an equivalent), and, where applicable, a correlation/request/session identifier sufficient to reconstruct a sequence of events (DS-OPS-002). Secrets and sensitive data are redacted at write time (DS-SCA-002) and defense-in-depth-checked at read time (DS-API-OPS-001). Retention and archival boundaries are documented and distinct from general application log retention (DS-OPS-002). If audit persistence itself becomes unavailable, the triggering action fails closed for security-sensitive actions (DS-SCA-001/015) rather than proceeding unaudited. This requirement does not mandate a specific storage vendor or cryptographic product; it is satisfiable by an append-only table pattern, a write-once log, or an equivalent mechanism chosen at implementation time. This requirement defines write-path and access-boundary architecture only; the deterministic behavior that actually detects direct storage corruption, truncation, reordering, or selective deletion is defined separately in DS-SCA-027.

Acceptance Criteria:

- No code path modifies or deletes an existing audit record; a correction is a new, linked record.
- No component outside the defined audit-write path can write to the audit log.
- Audit-log reads by a permission group other than the defined authorized readers are rejected (DS-SCA-005/006).
- A security-sensitive action that cannot be audited (audit persistence unavailable) does not proceed (fail-closed).
- Retention/archival boundaries are documented and distinct from general log retention.

Testing: Tamper-attempt adversarial test (attempt to modify/delete an existing audit record via every available code path, confirm rejection); unauthorized-read rejection test; fail-closed-on-audit-unavailability test for a security-sensitive action; retention-boundary audit.

DS-SCA-026 — Security Incident Response Lifecycle

Release Classification: Committed / MVP | **Governing Source:** SECURITY_RULES.md "Fail Closed," "Emergency Controls," "Broker Reconciliation," "Independent Review"; DS-SCA-001, DS-SCA-015, DS-SCA-016, DS-SCA-020

Description: DarkSage's security-incident lifecycle covers, at minimum: (1) **Detection** — a security-relevant anomaly is identified (audit-log monitoring, reconciliation failure, authentication anomaly, or user report); (2) **Classification/severity** — the incident is assigned a severity per DS-QA-019's defect-severity scale, extended to incidents (a Critical incident implies an active or imminent safety/financial/security breach); (3) **Containment** — the affected capability is isolated (e.g., the affected session, integration, or trading pathway is paused) without unnecessarily disabling unaffected capability; (4) **Session and credential revocation** — affected sessions (DS-SCA-004) and credentials (DS-SCA-002/003/013) are revoked/rotated; (5) **Trading disablement or fail-closed controls** — where the incident is trading-relevant, Emergency Stop/Flatten (DS-SCA-016) or an equivalent fail-closed control (DS-SCA-015) is applied; (6) **User notification** — the user is notified when the incident affects their account, data, or trading state, consistent with DS-OPS-003's understandable-communication standard; (7) **Evidence preservation** — audit records relevant to the incident (DS-SCA-025) are preserved and never pruned by a retention policy during an active investigation; (8) **Recovery/remediation** — the underlying cause is addressed before affected capability is restored; (9) **Validation before**

re-enabling — affected capability is validated (re-tested per the relevant DS-QA category) before being re-enabled, and live-trading capability specifically requires the same independent-review standard as DS-SCA-022's pre-live-trading gate before restoration; (10) **Post-incident review and audit record** — a post-incident review is recorded, referencing the preserved evidence, and closes the incident.

This lifecycle explicitly addresses: a compromised session (steps 3–4); compromised broker credentials (steps 3–4); suspected unauthorized trade activity (steps 3, 5, 7); a compromised update/dependency path (steps 3, 5, 8–9, cross-referencing DS-SCA-019); an audit-log integrity failure (steps 1, 7, detected and escalated per DS-SCA-027's verification contract, cross-referencing DS-SCA-025 — an integrity failure is itself Critical per DS-QA-019); malicious or corrupted external data/integration (steps 3, 5, 8, cross-referencing DS-SCA-014); and local-device compromise where detectable (steps 3–4, 6).

Owner/user authority is preserved throughout: no automated process disables the user's own account access as a side effect of incident response without notification per step 6, and re-enabling live-trading capability specifically requires the stronger validation step 9 describes — never an automated, unattended restoration. This requirement does not define operational staffing, on-call rotation, or legal/regulatory response policy; those remain out of scope (§14 Non-Goals).

Acceptance Criteria:

- Every listed incident type maps to at least the steps cited above; no incident type is handled ad hoc outside this lifecycle.
- Step 5 (trading disablement) is reachable independent of the code path that caused the incident, per DS-SCA-016's independent-reachability principle.
- Step 9 (validation before re-enabling) is never skipped for a live-trading-relevant incident, regardless of severity classification.
- Step 7 (evidence preservation) overrides normal retention policy for the duration of an active investigation.

Testing: Tabletop/simulated-incident walkthrough per listed incident type, confirming each applicable lifecycle step occurs in order; live-trading re-enablement test confirming step 9's validation gate cannot be bypassed.

DS-SCA-027 — Audit-Log Integrity Verification Contract

Release Classification: Committed / MVP | **Governing Source:** DS-SCA-025 (Committed, this document); SECURITY_RULES.md "Fail Closed," "Logging"

Description: This requirement defines the deterministic integrity-verification behavior DS-SCA-025's append-only/tamper-evident architecture requires to actually detect corruption, not merely to declare an intent — added in the DS-008-H2 repair to close that gap. The protected record set is every AuditLogEntry (DS-DB-020) written under DS-SCA-025's authorized-writer path, verified as a sequence (or an equivalent ordered set) rather than record-by-record in isolation, so a gap, reordering, or unauthorized insertion is detectable, not only an individual record's own tampering.

Integrity evidence: each record (or a periodic checkpoint over a range of records) carries deterministic evidence sufficient to detect: record modification (content no longer matches what was verifiably written); record reordering (sequence/ordering evidence no longer matches the expected order); truncation (an expected suffix of the sequence is missing); selective deletion (a

gap appears in the sequence/chain where a record should exist); and unauthorized insertion (a record exists that does not chain from/attach to the prior verified state). This is satisfiable by a cryptographically verifiable hash chain, an authenticated record structure, periodic signed checkpoints, or an equivalent deterministic mechanism — the exact mechanism is a DS-004/implementation decision, not fixed here.

Verification triggers: integrity verification runs, at minimum: (a) on application/backend service startup, before the audit log is treated as trustworthy for that session; (b) before any security-sensitive audit review (e.g., before GET /audit-log returns results for an administrative review, or before DS-SCA-026's Evidence Preservation step relies on the log); (c) on a scheduled/background cadence; and (d) immediately after any detected storage or persistence fault (e.g., an unclean shutdown, a database-consistency error, a disk-level fault) affecting the audit store.

Verification result states: each run produces one of: **Verified** (no integrity violation detected); **Violation Detected** (a specific modification/reordering/truncation/deletion/insertion was identified, with its location in the sequence); or **Verification Inconclusive** (the check itself could not complete, e.g., due to a concurrent fault) — Inconclusive is never silently treated as Verified.

Failure handling and escalation: a Violation Detected or Verification Inconclusive result triggers DS-SCA-026's Security Incident Response Lifecycle at Detection, classified at minimum High severity (Critical if the violation affects records relevant to an active investigation or live-trading authority) per DS-QA-019's severity scale, and is itself recorded as a new, forward-linked audit event — never a rewrite of the compromised range.

Fail-closed behavior: while a Violation Detected or Verification Inconclusive state is unresolved, any security-sensitive or trading-relevant operation whose authorization trail (DS-SCA-012) would depend on audit trustworthiness fails closed, consistent with DS-SCA-001/015/025 — the system does not proceed as though the audit log were trustworthy merely because the underlying feature would otherwise succeed.

Evidence preservation and repair boundaries: the compromised range (and its surrounding verified context) is preserved unmodified for investigation (DS-SCA-026 step 7). Repair/recovery may append new, clearly-marked records restoring forward integrity from the last known-good checkpoint, but shall never silently rebuild, rewrite, or replace the compromised range to make verification pass again without the violation first being escalated and recorded. Silently "fixing" a verification failure by regenerating history is explicitly prohibited.

Acceptance Criteria:

- Every listed detection category (modification, reordering, truncation, selective deletion, unauthorized insertion) is independently demonstrable via an adversarial test that directly manipulates the underlying storage, not only via an API-level modification attempt (which DS-SCA-025 already covers).
- Verification runs at every listed trigger point; a missing trigger (e.g., no startup verification) is itself a defect against this requirement.
- A Violation Detected or Inconclusive result never resolves silently to Verified without an explicit, recorded remediation step.
- No code path rewrites or regenerates a compromised range to suppress a verification failure without first escalating per DS-SCA-026.
- This requirement mandates no specific vendor or cryptographic product; a hash-chain, authenticated structure, or signed-checkpoint mechanism (or equivalent) each satisfy it.

Testing: Direct-storage-corruption adversarial test (bypassing the API to modify/delete/reorder/insert records directly) confirming detection at the next verification trigger; startup-verification test; pre-review-verification test; scheduled-verification test; post-fault-verification test; fail-closed test confirming a security-sensitive/trading-relevant operation is blocked while a violation is unresolved; evidence-preservation test confirming the compromised range is not altered during investigation; anti-silent-rebuild test confirming no code path regenerates history to pass verification without escalation.

14. Non-Goals

DS-008 does not: redesign any DS-004 architectural boundary or DS-006 API contract; select a specific authentication token mechanism (DS-006 Appendix A Open Question #2 remains open); commit live-trading timing beyond ROADMAP.md Phase 13/DS-EXE-007; weaken DS-001 §14's local-first-where-practical or privacy-by-design principles; introduce a new credential/identity architecture beyond the session lifecycle DS-006 already defines; or define operational staffing, on-call rotation, or legal/regulatory incident-response policy (DS-SCA-026 states the technical/architectural lifecycle only).

15. Dependencies

- DS-001, DS-002 (DS-SEC, DS-RSK, DS-EXE, DS-OPS, DS-DAT families), DS-004, DS-006
- SECURITY_RULES.md, TRADING_RULES.md, AGENTS.md
- .ai-workflow/AGENT_PROTOCOL.md (process precedent for DS-SCA-021)

16. Risks and Constraints

- **Governance-boundary-now / implementation-later pattern:** several requirements (DS-SCA-007, DS-SCA-012, DS-SCA-022) fix a Committed constraint on future Planned implementation, exactly mirroring DS-EXE-001/DS-API-EXE-001's established pattern, to avoid inventing unsupported Committed implementation scope while still closing the governance gap now.
- **Update-integrity grounding gap:** DS-SCA-019 has no direct DS-002/DS-004 requirement backing its Planned classification beyond general SECURITY_RULES.md principle — recorded in Appendix A rather than silently treated as fully grounded.
- **Classification discipline:** no requirement in this document promotes a Planned upstream item (DS-EXE-003 through 006, DS-ARC-021/022/024, DS-INT-003) to Committed; Committed requirements here trace only to already-Committed DS-002/DS-004/DS-006 sources or restate them without expanding scope.

17. Verification Approach

Each DS-SCA-NNN requirement states its own Testing. Document-level verification (unique-ID check, cross-reference consistency against DS-002/DS-004/DS-006/SECURITY_RULES.md, no Committed requirement depending on a Planned-only capability) recorded in .ai-workflow/HANDOFF.md.

18. References

- SECURITY_RULES.md, TRADING_RULES.md, AGENTS.md
- docs/codex/Volume-02-Product/requirements/DS-SEC-Security-and-Privacy.md, DS-RSK-Risk.md, DS-EXE-Execution-and-Broker.md, DS-OPS-Operations.md, DS-DAT-Data-Management.md
- docs/codex/Volume-04-Architecture/DS-004-Technical-Architecture.md
- docs/codex/Volume-06-API/DS-006-API-Specification.md
- docs/secret-scanning.md

Appendix A — Open Questions

1. **Authentication token mechanism** — unchanged from DS-006 Appendix A #2; DS-SCA-004/005 remain mechanism-agnostic pending that decision.
2. **Application update/distribution mechanism** — no DS-002/DS-004 requirement currently commits a specific mechanism (Electron auto-updater vs. manual installer distribution); DS-SCA-019 states the security obligation that applies once one is chosen. Recommend a future DS-004/DS-011 addition to formalize the mechanism itself.
3. **Governance-confirmation carryover** — the standing BLOCKERS.md items (ROADMAP.md phase boundaries as Codex release-scope authority; phase-mapping precision) apply identically to this document's Release Classification scheme and are not re-litigated here.